



LeetCode 26: Remove Duplicates from Sorted Array

1. Problem Title & Link

Title: LeetCode 26: Remove Duplicates from Sorted Array

Link: <https://leetcode.com/problems/remove-duplicates-from-sorted-array/>

2. Problem Statement (Short Summary)

Given a **sorted array** `nums`, remove the duplicates **in-place** such that each unique element appears only once.

Return the number of unique elements `k`.

After modification:

First `k` elements

should contain all unique values.

The remaining elements do not matter.

Example

Input:

`nums = [1,1,2]`

Output:

`k = 2`

Modified array:

`[1,2,_]`

(`_` means ignored value)

3. Examples (Input → Output)

Example 1

Input:

`nums = [1,1,2]`

Output:

`2`

Modified:

`[1,2,_]`

Example 2

Input:

`nums = [0,0,1,1,1,2,2,3,3,4]`

Output:

`5`

Modified:



[0,1,2,3,4,_,_,_,_]

Example 3

Input:

nums = [1]

Output:

1

4. Constraints

$1 \leq \text{nums.length} \leq 3 * 10^4$

$-100 \leq \text{nums}[i] \leq 100$

Important:

Array is sorted.

And:

Must modify in-place.

5. Core Concept (Pattern / Topic)

Two Pointers

Secondary Topics:

- In-Place Array Modification
- Sorted Array Processing

6. Thought Process (Step-by-Step Explanation)

Brute Force Idea

Create a new array.

Store only unique values.

Example:

[1,1,2]

New array:

[1,2]

Works.

Problem

Question explicitly asks:

In-place modification

Need to use the original array.



Important Observation

Array is sorted.

Example:

[0,0,1,1,1,2,2,3]

Duplicates always appear together.

Observe:

0 0

↑ ↑

Duplicate.

1 1 1

↑ ↑ ↑

Duplicate.

This makes detection easy.

Two Pointer Idea

Use:

read pointer

to scan array.

Use:

write pointer

to place unique elements.

Example

Input:

[1,1,2]

Initially:

write = 0

Read:

1

Already stored.

Next:



1

Duplicate.

Ignore.

Next:

2

New value.

Move write pointer:

write = 1

Store:

nums[1] = 2

Array becomes:

[1,2,2]

Unique count:

2

7. Visual / Intuition Diagram (ASCII Diagram)

Input:

[1,1,2]

W

R

[1,1,2]

Read next:

[1,1,2]

R

Duplicate.

Ignore.

Read next:

[1,1,2]

R

New value.

Move write:

W

[1,2,2]

Answer:

k = 2



8. Pseudocode (Language Independent)

```
write = 0

for read from 1 to n-1

    if nums[read] != nums[write]

        write++

        nums[write] = nums[read]

return write + 1
```

9. Code Implementation

Python

```
class Solution:
    def removeDuplicates(self, nums):

        write = 0

        for read in range(1, len(nums)):

            if nums[read] != nums[write]:

                write += 1

                nums[write] = nums[read]

        return write + 1
```

Java

```
class Solution {

    public int removeDuplicates(int[] nums) {

        int write = 0;

        for(int read = 1;
            read < nums.length;
            read++) {
```



```
        if(nums[read] != nums[write]) {  
            write++;  
            nums[write] = nums[read];  
        }  
    }  
    return write + 1;  
}
```

10. Time & Space Complexity

Time Complexity

$O(n)$

One traversal.

Space Complexity

$O(1)$

No extra array used.

11. Common Mistakes / Edge Cases

Mistake 1

Using extra array.

Question requires:

In-place modification.

Mistake 2

Comparing with previous index.

Wrong:

`nums[read]`

`!=`

`nums[read-1]`

Can work for counting but not for in-place placement.

Need:

`nums[write]`

Mistake 3



Returning:

write

instead of:

write + 1

Edge Cases

Single Element

Input:

[1]

Output:

1

All Unique

Input:

[1,2,3,4]

Output:

4

All Same

Input:

[5,5,5,5]

Output:

1

12. Detailed Dry Run

Input:

[0,0,1,1,1,2,2,3,3,4]

Initial:

write = 0

read	nums[read]	Action	write
1	0	Duplicate	0
2	1	Store	1
3	1	Duplicate	1
4	1	Duplicate	1
5	2	Store	2



6	2	Duplicate	2
7	3	Store	3
8	3	Duplicate	3
9	4	Store	4

Array:

[0,1,2,3,4,...]

Return:

5

13. Common Use Cases (Real-Life / Interview)

Data Cleaning

Removing duplicate records.

Database Optimization

Store only unique entries.

Log Processing

Remove repeated events.

Interview Purpose

Introduces:

Two Pointers

In-place modification

14. Common Traps (Important!)

Trap 1

Ignoring:

Array is sorted.

Sorting property makes the solution simple.

Trap 2

Creating new arrays.

Violates:

$O(1)$ extra space

requirement.

**Trap 3**

Returning wrong length.

Remember:

write points to index.

Length = write + 1

15. Builds To (Related LeetCode Problems)

LC 26 - Remove Duplicates from Sorted Array

↓

LC 27 - Remove Element

↓

LC 80 - Remove Duplicates II

↓

LC 283 - Move Zeroes

↓

LC 977 - Squares of Sorted Array

↓

LC 167 - Two Sum II

These are classic Two Pointer problems.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
HashSet	O(n)	O(n)	Poor
Extra Array	O(n)	O(n)	Fair
Two Pointers	O(n)	O(1)	Best

Approach 1: HashSet

Store unique values.

Easy but violates in-place requirement.

Approach 2: Extra Array

Collect unique values.

Still uses extra memory.

Approach 3: Two Pointers (Chosen)



Uses:

Read Pointer

Write Pointer

Most optimal.

17. Why This Solution Works (Short Intuition)

Because the array is sorted, duplicates always appear consecutively.

The read pointer scans every element, while the write pointer keeps track of where the next unique element should be placed.

Every unique value is written exactly once into the front portion of the array.

18. Variations / Follow-Up Questions

Follow-Up 1

Allow duplicates at most twice.

Related:

LC 80

Follow-Up 2

Remove a specific value.

Related:

LC 27

Follow-Up 3

Move all zeros to the end.

Related:

LC 283

Follow-Up 4

Count unique elements without modifying the array.

Interview Takeaway

This is one of the most important **Two Pointer** interview questions.

The core pattern is:

Read Pointer → scans data

Write Pointer → builds answer

Whenever you hear:



In-place

Sorted Array

Remove Elements

Compact Array

immediately think:

Two Pointers

This pattern appears repeatedly in Amazon, Zoho, TCS, Infosys, Wipro, Accenture, and product-company interviews.